PARTIE 1

Qu'est-ce qu'une API distante ?

Cours de Bachelor 3 — Développement Back-end

Spring Boot 4.1.1 · Java 25

Durée	Jour	Prérequis
2 h 10	**Jour 1 — matin**	**Bases de Java**

> **Version 2** — Prérequis alignés sur la version du cours : **Java 25** (et Spring Boot 4.1.1 à partir de la Partie 3).

1. Introduction

Objectifs de cette partie

À la fin de cette partie, vous serez capables de :

- Définir ce qu'est une API et une API distante.
- Comprendre pourquoi les API sont incontournables dans le développement moderne.
- Distinguer les différents types d'API (locale, distante, publique, privée).
- Effectuer votre premier appel à une API distante avec Postman ou curl.
- Lire et interpréter une réponse JSON.

Durée estimée

- Théorie : 1 h 30
- Pratique : 1 h 30

---

2. Concepts théoriques essentiels

2.1 Qu'est-ce qu'une API ?

API signifie *Application Programming Interface* — en français : **interface de programmation applicative**.

C'est un **contrat** entre deux logiciels. L'un expose des fonctionnalités, l'autre les consomme. Pensez-y comme au menu d'un restaurant : vous ne cuisinez pas vous-même, vous choisissez dans le menu (l'API), le serveur transmet votre commande à la cuisine (le serveur distant), et vous recevez votre plat (la réponse).

Analogie concrète :

```
Client  →  [Requête]  →  API  →  Serveur  →  [Réponse]  →  Client


CLIENT (Vous)
    ↓
    | [Requête/Commande] "Je voudrais un café"
    ↓
SERVEUR (API)
    ↓
    | Transmet la requête
    ↓
CUISINE (Traitement/Backend)
    ↓
    | Prépare selon la commande
    ↓
SERVEUR (API)
    ↓
    | [Réponse/Plat] "Voilà votre café chaud"
    ↓
CLIENT (Vous)
```

En informatique, cela donne :

```
App mobile → [GET/POST HTTP] → API REST → Backend → [JSON] → App mobile
```

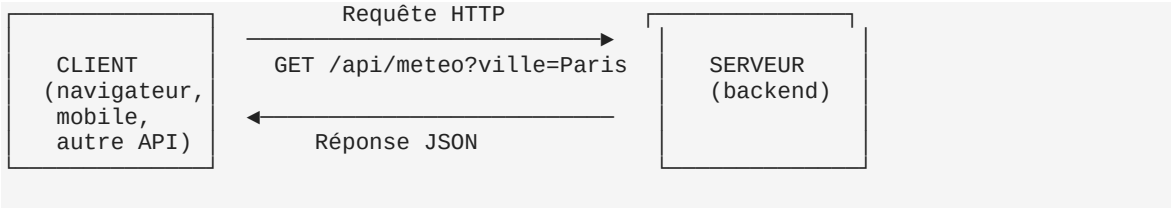
2.2 API locale vs API distante

Caractéristique	API locale	API distante
Emplacement	Même machine / même processus	Serveur distant accessible via le réseau
Communication	Appels de méthodes en mémoire	Protocole réseau (HTTP, HTTPS)
Exemple	`java.util.List` en Java	API météo, API Google Maps
Latence	Quasi nulle	Variable (réseau)
Format d'échange	Objets en mémoire	JSON, XML

API locale — quand vous appelez `list.add("Bonjour")` en Java, vous utilisez l'API de la classe `List`. Tout se passe en mémoire, sur votre machine.

API distante — quand votre application mobile demande la météo à un serveur, elle envoie une requête HTTP à travers le réseau et reçoit une réponse. C'est une API distante.

2.3 Schéma : communication client-serveur



Le client **ne sait pas** comment le serveur fonctionne en interne. Il connaît uniquement le **contrat** : quelle URL appeler, quels paramètres envoyer, quel format de réponse attendre. C'est le principe d'**encapsulation**.

2.4 Pourquoi les API distantes sont-elles partout ?

Les API distantes sont le ciment de l'internet moderne. Voici des exemples concrets :

Situation du quotidien	API distante utilisée
Vous consultez la météo sur votre téléphone	L'application appelle une API météo (OpenWeatherMap, etc.)
Vous payez en ligne	Le site appelle l'API de la passerelle de paiement (Stripe, PayPal)
Vous vous connectez avec Google	Le site utilise l'API OAuth de Google
Vous commandez un VTC	L'application appelle des API de géolocalisation et de tarification
Une application affiche une carte	Elle consomme l'API Google Maps ou OpenStreetMap

Avantages des API distantes :

- **Réutilisation** : une seule API peut servir une application web, mobile et desktop.
- **Séparation des responsabilités** : le frontend et le backend évoluent indépendamment.

- **Scalabilité** : le serveur API peut être dimensionné séparément.
- **Interopérabilité** : des applications écrites dans des langages différents communiquent via un format commun (JSON).

2.5 Les différents types d'API distantes

Type	Accès	Exemple
API publique (ouverte)	Accessible à tous, souvent avec une clé API	OpenWeatherMap, JSONPlaceholder
API privée (interne)	Réservée à l'entreprise	API interne de gestion des commandes
API partenaire	Accessible à des tiers autorisés	API bancaire (Open Banking)

2.6 Les formats d'échange de données

Quand deux applications communiquent via une API distante, elles ont besoin d'un format commun pour les données. Les deux formats les plus répandus sont **JSON** et **XML**.

JSON (JavaScript Object Notation)

C'est le format dominant aujourd'hui. Il est léger, lisible et facile à parser.

```
{
  "id": 1,
  "nom": "Dupont",
  "prenom": "Marie",
  "email": "marie.dupont@email.com",
  "age": 28
}
```

Règles JSON à retenir :

- Les clés sont toujours entre guillemets doubles.
- Les types possibles : `string`, `number`, `boolean`, `null`, `array`, `object`.
- Pas de commentaires.
- Les tableaux utilisent les crochets `[]`.

Exemple de tableau JSON :

```
{
  "utilisateurs": [
    { "id": 1, "nom": "Dupont" },
    { "id": 2, "nom": "Martin" }
  ]
}
```

XML (eXtensible Markup Language)

Plus ancien, plus verbeux, encore utilisé dans certains systèmes (SOAP, flux RSS).

```
<utilisateur>
  <id>1</id>
  <nom>Dupont</nom>
  <prenom>Marie</prenom>
  <email>marie.dupont@email.com</email>
  <age>28</age>
</utilisateur>
```

Comparaison JSON vs XML

Critère	JSON	XML
Lisibilité	Très lisible	Verbeux
Poids	Léger	Plus lourd
Parsing en Java	Simple (Jackson, Gson)	Plus complexe (JAXB, DOM)
Usage actuel	Standard pour les API REST	Systèmes legacy, SOAP

Dans ce cours, nous utiliserons exclusivement **JSON**.

2.7 Le protocole HTTP en bref

Les API distantes les plus courantes s'appuient sur le protocole **HTTP** (HyperText Transfer Protocol). Voici les bases à connaître.

Structure d'une requête HTTP

```
[MÉTHODE] [URL] HTTP/1.1
[En-têtes]

[Corps (optionnel)]
```

Exemple concret :

```
GET /api/utilisateurs/1 HTTP/1.1
Host: monserveur.com
Accept: application/json
```

Les méthodes HTTP principales

Méthode	Action	Idempotente ?
`GET`	Lire une ressource	Oui
`POST`	Créer une ressource	Non
`PUT`	Remplacer une ressource	Oui
`DELETE`	Supprimer une ressource	Oui
`PATCH`	Modifier partiellement	Non

Idempotent signifie que répéter la même requête plusieurs fois produit le même résultat.

Les codes de statut HTTP

Code	Signification	Quand ?
`200 OK`	Succès	Lecture réussie
`201 Created`	Ressource créée	Après un POST réussi
`204 No Content`	Succès sans contenu	Après un DELETE réussi
`301 Moved Permanently`	L'endpoint s'est déplacé définitivement	/api/v1/produits → /api/v2/produits
`302 Found`	Redirection temporaire (rare en API)	Redirection provisoire
`304 Not Modified`	Déjà couvert	Cache HTTP
`308 Permanent Redirect`	Redirection permanente (préserve la méthode)	Remplace 301 en API moderne

Code	Signification	Quand ?
`400 Bad Request`	Requête mal formée	Paramètres invalides
`401 Unauthorized`	Non authentifié	Clé API manquante
`403 Forbidden`	Accès interdit	Droits insuffisants
`404 Not Found`	Ressource introuvable	Mauvais identifiant
`500 Internal Server Error`	Erreur serveur	Bug côté serveur

Astuce mnémotechnique :

- **2xx** → Tout va bien
- **3xx** → Redirection ou ressource non modifiée (caching)
- **4xx** → Le client a fait une erreur
- **5xx** → Le serveur a un problème

---

3. Mise en pratique guidée

3.1 Prérequis

Avant de commencer, assurez-vous d'avoir installé :

- **Java 25**, la version utilisée dans tout le cours (`java -version` dans le terminal). Le code de cette partie fonctionne dès Java 17.
- **Postman** (téléchargeable gratuitement) ou **curl** (intégré à Linux/macOS, disponible sur Windows)
- Un **navigateur web** récent

3.2 Découverte de JSONPlaceholder

JSONPlaceholder est une API publique gratuite parfaite pour apprendre. Elle simule un backend avec des utilisateurs, des posts, des commentaires, etc.

URL de base : `https://jsonplaceholder.typicode.com`

Étape 1 : Premier appel avec le navigateur

Ouvrez votre navigateur et tapez dans la barre d'adresse :

```
https://jsonplaceholder.typicode.com/users/1
```

Vous devriez voir apparaître un objet JSON représentant un utilisateur. Le navigateur a envoyé une requête `GET` et a reçu une réponse JSON.

Étape 2 : Premier appel avec curl

Ouvrez un terminal et tapez :

```
curl https://jsonplaceholder.typicode.com/users/1
```

Vous obtenez le même résultat, mais cette fois dans le terminal.

Pour voir les en-têtes HTTP de la réponse :

```
curl -i https://jsonplaceholder.typicode.com/users/1
```

Observez les en-têtes : `Content-Type: application/json`, le code de statut `200 OK`, etc.

Étape 3 : Premier appel avec Postman

1. Ouvrez Postman.
2. Créez une nouvelle requête.
3. Sélectionnez la méthode **GET**.
4. Entrez l'URL : `https://jsonplaceholder.typicode.com/users/1`
5. Cliquez sur **Send**.
6. Observez la réponse dans le panneau inférieur : le corps JSON, le code de statut (`200 OK`), le temps de réponse et la taille.

3.3 Explorer les endpoints disponibles

Testez les URLs suivantes (méthode GET) et observez les réponses :

URL	Ce qu'elle retourne
`/users`	Liste de tous les utilisateurs
`/users/1`	Un utilisateur spécifique
`/posts`	Liste de tous les posts
`/posts/1`	Un post spécifique
`/posts/1/comments`	Les commentaires du post n°1
`/posts?userId=1`	Les posts de l'utilisateur n°1

3.4 Simuler une création avec POST

Dans Postman :

1. Changez la méthode en **POST**.
2. URL : `https://jsonplaceholder.typicode.com/posts`
3. Allez dans l'onglet **Body** → sélectionnez **raw** → choisissez **JSON**.
4. Entrez ce corps :

```
{
  "title": "Mon premier post",
  "body": "Ceci est le contenu de mon post créé via API.",
  "userId": 1
}
```

5. Cliquez sur **Send**.
6. Observez la réponse : code `201 Created`, et la ressource créée avec un `id` généré.

Avec curl :

```
curl -X POST https://jsonplaceholder.typicode.com/posts \
  -H "Content-Type: application/json" \
  -d '{
    "title": "Mon premier post",
    "body": "Ceci est le contenu de mon post créé via API.",
```

```
    "userId": 1
  }'
```

Décortiquons cette commande :

- `-X POST` : spécifie la méthode HTTP.
- `-H "Content-Type: application/json"` : indique que le corps est en JSON.
- `-d '...'` : le corps de la requête.

3.5 Lire une réponse JSON en Java

Créons un petit programme Java qui appelle une API distante et affiche le résultat. Ce code utilise `HttpClient`, disponible nativement depuis Java 11.

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class PremierAppelApi {

    public static void main(String[] args) throws Exception {

        // 1. Créer un client HTTP
        HttpClient client = HttpClient.newHttpClient();

        // 2. Construire la requête GET
        HttpRequest requete = HttpRequest.newBuilder()
                .uri(URI.create("https://jsonplaceholder.typicode.com/users/1"))
                .GET()   // GET est la méthode par défaut, mais on l'explicite pour
la clarté
                .header("Accept", "application/json")
                .build();

        // 3. Envoyer la requête et récupérer la réponse
        HttpResponse<String> reponse = client.send(
                requete,
                HttpResponse.BodyHandlers.ofString()
        );

        // 4. Afficher le code de statut
        System.out.println("Code HTTP : " + reponse.statusCode());

        // 5. Afficher le corps de la réponse
        System.out.println("Réponse :");
        System.out.println(reponse.body());
    }
}
```

Compilation et exécution :

```
javac PremierAppelApi.java
java PremierAppelApi (sans l'extension .class!)
```

Résultat attendu :

```
Code HTTP : 200
Réponse :
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  ...
}
```

Explication du code :

- `HttpClient` : le client HTTP intégré à Java, capable d'envoyer des requêtes synchrones et asynchrones.
- `HttpRequest` : on construit la requête en précisant l'URL, la méthode et les en-têtes.
- `HttpResponse<String>` : la réponse contient le code de statut, les en-têtes et le corps sous forme de `String`.
- `BodyHandlers.ofString()` : indique qu'on veut le corps de la réponse sous forme de chaîne de caractères.

3.6 Envoyer une requête POST en Java

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class CreerPost {

    public static void main(String[] args) throws Exception {

        // Corps de la requête en JSON
        String corpsJson = """
                {
                    "title": "Mon post depuis Java",
                    "body": "Contenu envoyé via HttpClient.",
                    "userId": 1
                }
                """;

        HttpClient client = HttpClient.newHttpClient();

        // Construire une requête POST avec un corps JSON
        HttpRequest requete = HttpRequest.newBuilder()
                .uri(URI.create("https://jsonplaceholder.typicode.com/posts"))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(corpsJson))
                .build();

        HttpResponse<String> reponse = client.send(
                requete,
                HttpResponse.BodyHandlers.ofString()
        );

        System.out.println("Code HTTP : " + reponse.statusCode());
        System.out.println("Réponse : " + reponse.body());
    }
}
```

Points à noter :

- Les **Text Blocks** (`"""..."""`) de Java 15+ permettent d'écrire du JSON multiligne proprement.
- `BodyPublishers.ofString(corpsJson)` transforme la chaîne en corps de requête.
- L'en-tête `Content-Type: application/json` est indispensable pour que le serveur interprète correctement le corps.

4. Exercices pratiques

Exercice 1 — Exploration d'API (Facile)

Objectif : Se familiariser avec les appels API et la lecture de JSON.

Consigne : En utilisant Postman ou curl, répondez aux questions suivantes en interrogeant `https://jsonplaceholder.typicode.com` :

1. Combien d'utilisateurs sont retournés par `/users` ?
2. Quel est le nom de l'utilisateur ayant l'`id` 5 ?
3. Combien de posts l'utilisateur n°1 a-t-il écrits ? (Indice : filtrez avec `?userId=1`)
4. Quel est le titre du post ayant l'`id` 42 ?
5. Listez les commentaires du post n°3. Combien y en a-t-il ?

Résultat attendu : Un document texte avec les réponses et les URLs utilisées pour chaque question.

Travail en binôme : Un étudiant dicte l'URL, l'autre l'exécute et vérifie la réponse.

Exercice 2 — Client HTTP en Java (Intermédiaire)

Objectif : Écrire un programme Java qui consomme une API distante.

Consigne : Créez une classe Java `ExplorateurApi` qui :

1. Récupère la liste de tous les posts (`GET /posts`).
2. Affiche le nombre total de posts.
3. Récupère un post spécifique par son `id` (passé en argument du programme).
4. Affiche le titre et le corps de ce post.
5. Gère le cas où le post n'existe pas (code 404).

Squelette de départ :

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class ExplorateurApi {

    private static final String BASE_URL = "https://jsonplaceholder.typicode.com";
    private static final HttpClient client = HttpClient.newHttpClient();

    public static void main(String[] args) throws Exception {
        // TODO 1 : Récupérer tous les posts et compter le nombre de lignes "id"
        // TODO 2 : Récupérer le post dont l'id est passé en argument
        // TODO 3 : Gérer le cas 404
    }

    /**
     * Envoie une requête GET et retourne la réponse.
     */
    private static HttpResponse<String> get(String chemin) throws Exception {
        HttpRequest requete = HttpRequest.newBuilder()
                .uri(URI.create(BASE_URL + chemin))
                .GET()
                .build();
```

```
        return client.send(requete, HttpResponse.BodyHandlers.ofString());
    }
}
```

Résultat attendu :

```
Nombre total de posts : 100
--- Post n°42 ---
Titre : provident id voluptas
Corps : sapiente assumenda molestiae atque...


--- Post n°999 ---
Erreur : post introuvable (code 404)
```

Exercice 3 — Simuler un CRUD complet (Avancé)

Objectif : Comprendre le cycle de vie d'une ressource via une API distante.

Consigne : Créez une classe Java `CrudPosts` qui enchaîne les opérations suivantes sur `https://jsonplaceholder.typicode.com/posts` :

1. **CREATE** — Créer un nouveau post (méthode POST) avec un titre et un corps de votre choix.
2. **READ** — Lire le post qui vient d'être créé (méthode GET avec l'`id` retourné).
3. **UPDATE** — Modifier le titre du post (méthode PUT).
4. **DELETE** — Supprimer le post (méthode DELETE).
5. Pour chaque opération, affichez la méthode HTTP, l'URL, le code de réponse et le corps.

Contraintes :

- Créez une méthode utilitaire par type de requête (`get`, `post`, `put`, `delete`).
- Utilisez les **Text Blocks** Java pour les corps JSON.
- Affichez clairement chaque étape avec un séparateur visuel.

Résultat attendu :

```
=== CREATE ===
POST /posts → 201
Réponse : {"id":101,"title":"Mon post",...}

=== READ ===
GET /posts/101 → 200
Réponse : {"id":101,"title":"Mon post",...}

=== UPDATE ===
PUT /posts/101 → 200
Réponse : {"id":101,"title":"Titre modifié",...}

=== DELETE ===
DELETE /posts/101 → 200
Réponse : {}
```

Travail en binôme : Un étudiant code les méthodes CREATE et READ, l'autre code UPDATE et DELETE. Assemblez le tout ensemble.

5. Résumé des points clés

Ce qu'il faut retenir :

- Une **API** est un contrat entre deux logiciels : l'un expose des fonctionnalités, l'autre les consomme.
- Une **API distante** communique à travers le réseau via le protocole HTTP.
- Le format d'échange standard est **JSON** : léger, lisible et universel.
- Les **méthodes HTTP** (GET, POST, PUT, DELETE) correspondent aux opérations de lecture, création, modification et suppression.
- Les **codes de statut** (2xx, 4xx, 5xx) indiquent le résultat de la requête.
- Java intègre un **HttpClient** natif (depuis Java 11) pour consommer des API distantes.
- **Postman** et **curl** sont des outils indispensables pour tester et explorer les API.

Vocabulaire à maîtriser :

Terme	Définition
API	Interface de programmation entre deux logiciels
Endpoint	URL spécifique exposée par une API
Requête	Message envoyé par le client au serveur
Réponse	Message retourné par le serveur au client
JSON	Format de données texte, léger et structuré
Code de statut	Nombre indiquant le résultat d'une requête HTTP
Client HTTP	Programme ou outil qui envoie des requêtes HTTP

6. Pour aller plus loin

> - Explorer d'autres API publiques (liste de centaines d'API gratuites)
> - Tester l'API GitHub avec `curl` (exemples détaillés et réponses attendues)
> - Comprendre les en-têtes HTTP courants (`Authorization`, `Cache-Control`, `Accept-Language`, etc.)
> - Ressources complémentaires pour s'entraîner